GUIDES

Set up Claude Code in a monorepo or large codebase

Configure Claude Code for monorepos and large single-tree codebases with nested CLAUDE.md files, sparse worktrees, code intelligence, and per-package skills so Claude stays focused on the code you're working in.

A large codebase can be one repository with millions of lines or a monorepo with many packages. Claude Code works at any size, but as the codebase grows, the defaults tuned for smaller projects can fill the context window with instructions and file reads unrelated to the task, costing tokens and degrading Claude's performance.

This guide shows individual developers and engineering teams how to scope Claude to the part of the codebase a task touches. Each section notes whether a setting is personal to your machine or committed to the repository.

What this guide covers

The **table below** lists each setting and what it accomplishes. The **file tree after it** is the example monorepo every code sample on this page refers to.

Settings on this page

Each setting below is independent. They layer rather than replace each other, so apply whichever fit your repository. **Choose where to start Claude** determines where your settings files live, so read it first. **Put it together** shows all of them combined.

I want to	Use
Load only the conventions for the code you touch, instead of one root file covering every subsystem	Per-directory **CLAUDE.md files**
Exclude CLAUDE.md files for packages you never work in	`claudeMdExcludes`
Block Claude from opening build output, generated code, and vendored dependencies	`Read` **deny rules** in `permissions.deny`

I want to	Use
Find a symbol's definition or callers through the language server instead of scanning files	A **code intelligence plugin**
Check out only the directories a task needs when Claude creates a worktree	`worktree.sparsePaths`
Read and edit a sibling package or another repository from the same session	`--add-dir` or `additionalDirectories`
Give Claude procedures specific to one area that load only when relevant	Per-directory **skills**
Replace many per-directory CLAUDE.md files with one set of conventions everyone installs	A **plugin** in an internal marketplace

For workflow techniques that keep context small in any repository, such as **running exploration in a subagent** so file reads stay out of the main conversation, see **Best practices for Claude Code**. To roll out a baseline configuration to every developer in your organization, see **Set up Claude Code for your organization**.

The example monorepo

The examples throughout this page refer to a monorepo with three packages. The same patterns work in a large single-tree codebase: where an example uses `packages/api/`, substitute your own subsystem directory such as `src/backend/` or `lib/core/`.

```
monorepo/
  CLAUDE.md                    # root instructions
  packages/
    api/
      CLAUDE.md                # API-specific instructions
      .claude/skills/
      src/
    web/
      CLAUDE.md                # frontend-specific instructions
      .claude/skills/
      src/
    shared/
      CLAUDE.md                # shared library instructions
      src/
```

Choose where to start Claude

Where you launch `claude` determines which files Claude can read and edit without an additional permission grant, which CLAUDE.md files load into context at startup, and which project settings apply.

Start from	File access	CLAUDE.md loaded at launch	Use when
Repository root	Every file	Root only; subdirectory files load on demand when Claude reads there	Tasks span multiple packages or subsystems
A subdirectory	That subtree only, until you grant more	That directory's plus every ancestor's	Work is scoped to one package or subsystem

Project settings in `.claude/settings.json` load only from your starting directory and are not inherited from parent directories the way CLAUDE.md files are: a `.claude/settings.json` at the repository root applies only when you start from the root.

Each section below states whether its settings file belongs at the repository root or in the subdirectory you start from, and whether it is committed or kept local.

Layer CLAUDE.md files by directory

In a large codebase, a single CLAUDE.md at the repository root tends to either grow to cover every subsystem's conventions, costing context on instructions unrelated to the current task, or stay too generic to be useful. Splitting instructions across per-directory files means Claude loads repository-wide rules plus only the conventions for the code you're working in.

Claude Code loads every **CLAUDE.md** file from your working directory and every parent directory at launch, then loads each subdirectory's file on demand when it reads files there. A root file sets repository-wide rules and each subdirectory adds its own.

A common split is two levels:

- **Root** `CLAUDE.md`: instructions that apply everywhere, such as coding standards, commit conventions, and repository layout
- **Per-subdirectory** `CLAUDE.md`: conventions specific to that area's stack. In a monorepo that's one per package. In a large single tree it's one per subsystem such as `src/db/` or `src/api/`

Commit these files to the repository so teammates inherit them. Each directory's owner typically

maintains its file.

The root `CLAUDE.md` orients Claude to the repository structure:

CLAUDE.md

```
This is a monorepo with three packages under packages/:

- packages/api: Node.js REST API with Express, TypeScript, and PostgreSQL
- packages/web: React frontend with Vite, TypeScript, and TailwindCSS
- packages/shared: shared TypeScript utilities used by both api and web

Run commands from the package directory, not the monorepo root.
Each package has its own tsconfig.json, package.json, and test suite.
```

Each subdirectory's `CLAUDE.md`, here `packages/api/CLAUDE.md`, adds context specific to that area's stack:

packages/api/CLAUDE.md

```
This package is the REST API server.

- Run tests: `npm test` (uses Vitest)
- Run dev server: `npm run dev` (port 3001)
- Database migrations: `npm run migrate`
- Environment variables: copy `.env.example` to `.env`

API routes are in src/routes/. Each route file exports an Express router.
Database queries use Knex in src/db/. Never write raw SQL strings in route
handlers.
```

When you start Claude from `packages/api/`, it loads both `packages/api/CLAUDE.md` and the root `CLAUDE.md`. Claude sees the local instructions alongside the repository-wide rules, with no instructions from `packages/web/` in context. The same holds for any subdirectory in a non-monorepo tree.

A few ways to keep the files current as the codebase and models change:

- **Review in pull requests**: treat CLAUDE.md edits like any other documentation change so conventions track the code
- **Revisit after major model releases**: instructions that worked around an older model's limitation

may become overhead once a newer model handles the case on its own. For example, a rule that forces single-file refactors can be deleted once the limitation is gone

- **Add a Stop hook that proposes updates**: a `Stop` **hook** receives the path to the session transcript when Claude finishes responding, so a script can review the session and propose CLAUDE.md updates while the gap it exposed is fresh

For more on how CLAUDE.md files load and interact, see **Memory and project instructions**.

Choose between per-directory CLAUDE.md and path-scoped rules

Per-directory `CLAUDE.md` files and **path-scoped rules** under `.claude/rules/` both let you target instructions to part of the tree. They differ in where the file lives and when it loads.

Approach	File location	Loads when	Use when
Per-directory `CLAUDE.md`	Inside the directory, alongside its code	At launch when started from that directory, or on demand when Claude reads a file there	Directory owners maintain their own conventions; instructions are versioned with the code
Path-scoped rule in `.claude/rules/`	Central `.claude/` at the repo root	When Claude works with a file matching the rule's `paths:` glob	You want all conventions in one place, or the same rule applies to many scattered paths

For a comparison that also covers skills, see **Compare similar features**.

Exclude irrelevant CLAUDE.md files

When you start Claude from the repository root, each subdirectory's CLAUDE.md loads as soon as Claude reads a file in that directory. The `claudeMdExcludes` setting skips specific files by path or glob pattern so they never load.

Use this for directories you never work in, such as other teams' packages, legacy code, or vendored subtrees. The exclusion list is static, not a per-task switch. To focus on one package today and another tomorrow, **start Claude from that package's directory** instead of editing exclusions.

If you only want these exclusions for yourself, put the setting in `.claude/settings.local.json`. Claude Code gitignores that file when it creates it; since you're creating it by hand here, add it to your gitignore. Patterns use glob syntax matched against absolute file paths, so start relative-style patterns with `**/` to match anywhere in the tree. The example below excludes packages owned by other teams:

.claude/settings.local.json

```
{
  "claudeMdExcludes": [
    "**/packages/admin-dashboard/**",
    "**/packages/legacy-*/**"
  ]
}
```

This skips every CLAUDE.md and rules file under those packages. The root CLAUDE.md and the packages you do work in still load normally.

These patterns cover other common cases:

- `"**/packages/*/CLAUDE.md"`: excludes every package's CLAUDE.md while keeping the root
- `"**/packages/web/**"`: excludes everything under the web package, including rules
- `"/home/user/monorepo/legacy/CLAUDE.md"`: excludes one specific file by absolute path

Managed policy CLAUDE.md files cannot be excluded, so organization-wide instructions always apply. You can set `claudeMdExcludes` at any **settings scope**: user, project, local, or managed. Arrays merge across scopes, so a team can set project-level defaults while individuals add local overrides.

For the full exclusion documentation, see **Exclude specific CLAUDE.md files**.

Reduce what Claude reads

Instructions are only part of what ends up in Claude's context. File reads are another cost that grows with the codebase. The settings below block reads of irrelevant paths and replace exhaustive file scans with language-server lookups.

Block reads of generated and vendored code

Claude's content searches respect `.gitignore` by default, so paths already listed there, such as `node_modules/`, `dist/`, and `build/`, stay out of search results without additional configuration.

For paths that are checked in, such as a vendored SDK or committed generated code, add `Read` deny rules in `permissions.deny` to block Claude from opening those files even when a search lists them.

To apply these exclusions for everyone working in the repository, commit them to `.claude/settings.json`. To keep them personal, use `.claude/settings.local.json` instead. Like other project settings on this page, these files load only from your starting directory. Place them at the repository root if you start Claude there, or in each package's `.claude/` if you start from subdirectories. To enforce the same deny rules in every session regardless of starting directory, set them in **managed settings**, which user and project settings cannot override.

The example below blocks build artifacts and a vendored SDK:

.claude/settings.json

```
{
  "permissions": {
    "deny": [
      "Read(./**/dist/**)",
      "Read(./**/build/**)",
      "Read(./**/*.generated.*)",
      "Read(./vendor/**)"
    ]
  }
}
```

Deny rules cover Claude's built-in file tools and recognized Bash file commands, including `cat`, `head`, `grep`, and `find`, when a denied path is passed as an argument. They do not filter denied paths out of a recursive search's output, and they do not cover arbitrary subprocesses that open files themselves. For the full pattern syntax, see **Read and Edit permission rules**.

Reduce file reads with code intelligence

In a large codebase, finding where a symbol is defined or used can cost many file reads and grep calls. **Code intelligence plugins** connect Claude to a language server so it can jump to definitions, find references, and surface type errors directly instead of scanning the tree.

The official marketplace has plugins for TypeScript, Python, Go, Rust, and other common languages. The example below installs the TypeScript plugin:

```
/plugin install typescript-lsp@claude-plugins-official
```

To enable a plugin for everyone in the repository rather than installing it yourself, add it to the

`enabledPlugins` **project setting**.

Code intelligence plugins require the language's language server binary on each developer's machine. See **which binary each language requires**. Installing from the official marketplace requires network access to GitHub, where the marketplace is hosted. On a restricted network, **add the marketplace from an internal Git host or local path** instead.

This pairs well with `claudeMdExcludes` and the `Read` deny rules above. Those keep irrelevant content out of context, and code intelligence keeps Claude from reading through what remains to locate a definition.

Scope worktrees and file access

These settings control what's on disk in worktrees and which directories Claude can read and write beyond your starting point.

Check out only the directories you need

The `--worktree` flag starts a session in a new git worktree so changes stay isolated from your main checkout. By default it checks out the entire repository. In a large repository, the `worktree.sparsePaths` setting uses git sparse-checkout to write only the listed directories plus root-level files to disk, so worktrees start faster and use less space.

If everyone working in this directory needs the same paths, commit the setting to `.claude/settings.json`. To add paths for yourself, use `.claude/settings.local.json`: the lists merge across scopes, so a local file can add paths to the committed list but not remove them. The example below shows the committed file:

.claude/settings.json

```
{
  "worktree": {
    "sparsePaths": [
      ".claude",
      "packages/api",
      "packages/shared"
    ]
  }
}
```

When Claude creates a worktree, it checks out only `.claude/`, `packages/api/`, and `packages/shared/` instead of the full tree. Paths in `sparsePaths` are relative to the repository root, regardless of which subdirectory you start Claude from. Any directory paths work here, not only package roots.

This is particularly useful for **subagent worktree isolation**. Subagents are parallel Claude instances spawned for subtasks, and each one that runs in a worktree gets a lightweight checkout instead of the full tree. All worktrees in a session share the same `sparsePaths`, so if one subagent needs `packages/api/` and another needs `packages/web/`, list both.

List directories in `sparsePaths`, not individual files. Root-level files like `package.json`, `tsconfig.base.json`, and lock files are always checked out alongside the directories you list. Root-level directories are not, so include `.claude` in the list if you want the repository root's `.claude/settings.json`, `.claude/rules/`, or `.claude/skills/` available inside the worktree.

To avoid duplicating large directories like `node_modules` across worktrees, pair `sparsePaths` with `symlinkDirectories` in the same `.claude/settings.json`:

.claude/settings.json

```
{
  "worktree": {
    "sparsePaths": [
      ".claude",
      "packages/api",
      "packages/shared"
    ],
    "symlinkDirectories": [
      "node_modules"
    ]
  }
}
```

This creates a symlink from each worktree's `node_modules/` back to the main repository's copy rather than duplicating it on disk.

The `sparsePaths` and `symlinkDirectories` settings are read from your starting directory before the worktree is created. After creation, the session's working directory is the worktree root, not the subdirectory you launched from. Project settings inside the worktree therefore load from the worktree root's `.claude/settings.json`, the checked-out copy of the repository root's file. Put any other settings you need inside worktrees, such as permission rules or hooks, in the repository root's `.claude/settings.json`.

For the full worktree settings reference, see **Worktree settings**.

Grant access across packages or repositories

This section applies when you start Claude from a subdirectory, or when a task spans multiple checkouts. If you start from the repository root in a single large tree, Claude already has access to every file and you can skip this.

When you start Claude from `packages/api/`, it can read and write files within that directory. If a task requires changes across packages, such as updating a shared type that both `api` and `web` import, you need to grant access to the sibling directory. The same mechanism grants access to a separately-checked-out repository.

The `additionalDirectories` setting in `.claude/settings.json` gives Claude access to directories outside the working directory. The example below grants access to two sibling packages:

.claude/settings.json

```
{
  "permissions": {
    "additionalDirectories": [
      "../shared",
      "../web"
    ]
  }
}
```

Relative paths resolve against the directory you start Claude from. With this configuration, Claude can read and edit files in `packages/shared/` and `packages/web/` while working from `packages/api/`.

You can also grant access at runtime without editing settings by passing `--add-dir` when you start Claude:

```
claude --add-dir ../shared
```

However you add a directory, Claude can read and edit files in it. Whether the directory's CLAUDE.md, `.claude/rules/` files, and skills also load depends on how you added it:

Added with	Loads CLAUDE.md and rules	Loads skills
`additionalDirectories` setting	Never	Never
`--add-dir` flag or `/add-dir` command	Only with the environment variable below	Yes

To load CLAUDE.md and rules files from a directory added with `--add-dir` or `/add-dir`, set the `CLAUDE_CODE_ADDITIONAL_DIRECTORIES_CLAUDE_MD` environment variable:

```
CLAUDE_CODE_ADDITIONAL_DIRECTORIES_CLAUDE_MD=1 claude --add-dir
../shared
```

The environment variable has no effect on directories listed in the `additionalDirectories` setting. See **Load from additional directories** for details.

For sibling directories that everyone in this area needs, commit `additionalDirectories` to `.claude/settings.json`. For a personal selection or one-off access, use `.claude/settings.local.json` or pass `--add-dir` at launch.

Add per-directory skills

Any subdirectory can define **skills** scoped to its own stack. A skill loads on demand when Claude determines it's relevant, so API-specific tooling doesn't consume context during frontend work.

Skills live under `.claude/skills/` inside the directory. Commit them alongside that area's code so anyone who clones the repository gets them. In a monorepo this can be one set of skills per package. In a large single-tree codebase it's one set per subsystem such as `src/db/.claude/skills/`.

Create a skill directory inside the subdirectory:

```
mkdir -p packages/api/.claude/skills/api-testing
```

Then write `SKILL.md` inside that directory, here `packages/api/.claude/skills/api-testing/SKILL.md`. This example teaches Claude the API package's testing patterns:

packages/api/.claude/skills/api-testing/SKILL.md

```
---
name: api-testing
description: Testing patterns for the API package. Use when writing or
modifying tests in packages/api/.
---

## Test structure

Tests are in `src/__tests__/` mirroring the `src/` directory structure.
Each route file has a corresponding `.test.ts` file.

## Running tests

- All tests: `npm test`
- Single file: `npm test -- src/__tests__/routes/users.test.ts`
- Watch mode: `npm test -- --watch`

## Test utilities

- `src/__tests__/helpers/db.ts`: provides `setupTestDb()` and
`teardownTestDb()` for database tests
- `src/__tests__/helpers/auth.ts`: provides `createTestUser()` and
`getAuthToken()` for authenticated endpoints

## Patterns

- Use `supertest` for HTTP assertions, not raw fetch
- Always wrap database tests in a transaction that rolls back
- Mock external services in `src/__tests__/mocks/`
```

A different subdirectory holds different skills the same way: `packages/web/.claude/skills/component-patterns/` describes the frontend's component conventions instead of testing. When Claude works on a file in `packages/api/`, it loads the api-testing skill. When it works in `packages/web/`, it loads component-patterns instead. Neither directory's skills load during the other's tasks.

You can also scope a skill by file pattern instead of by placement. The **`paths` frontmatter field** takes glob patterns, and Claude loads the skill automatically only when it works with matching files. Use

this for a skill that lives in the repository root's `.claude/skills/` but applies only to certain files wherever they appear, such as a database-migration skill scoped to `**/migrations/**`.

For more on creating and organizing skills, see **Skills**.

Keep skills discoverable

With skills spread across many directories, the list Claude chooses from can grow large. Claude picks a skill by reading every discovered skill's name and description, and only the chosen skill's full content loads into context. This section covers how to keep that list small and write descriptions that survive shortening.

Which skills are in scope depends on where you start Claude:

- **From a subdirectory such as** `packages/api/`: skills from that directory, every parent up to the repository root, and the user and enterprise levels
- **From the repository root**: skills from every subdirectory Claude touches during the session, which can accumulate into the hundreds
- **After adding a sibling with** `--add-dir`: that sibling's skills load too. The `additionalDirectories` setting grants file access only and does not load skills

Names always load, but **descriptions are shortened when there are many**, which can strip the keywords Claude uses to decide whether a skill applies. Keep descriptions short and lead with words a request would contain, like "writing or modifying tests in `packages/api/`".

For skills that many directories share, such as PR conventions or a deploy checklist, place them in the repository root's `.claude/skills/` so they load from any starting directory. When shared skills need their own version history or must work across repositories, package them as a **plugin** instead. Plugin skills use a `plugin-name:skill-name` namespace, so they never collide with per-directory skills. A platform team can version and update them in one place.

To find which skills go unused, enable the OpenTelemetry **logs exporter** and set `OTEL_LOG_TOOL_DETAILS=1` so skill names are recorded verbatim instead of redacted. The `skill_activated` **event** records every invocation in its `skill.name` attribute, and `invocation_trigger` records whether a command, Claude, or a nested skill invoked it, which tells you what to consolidate or retire.

Centralize conventions when layering stops scaling

Per-directory CLAUDE.md files can become hard to govern as the codebase grows. Conventions drift, files go stale, and no one owns the root. Solving that typically falls to the team that maintains the repository's Claude Code setup rather than to each developer working in their own area.

Move conventions and reference content out of always-loaded CLAUDE.md and into mechanisms that load on demand:

- **Skills**: reference material Claude loads only when relevant to the task
- **Plugins**: versioned bundles of skills, hooks, and commands that a platform team owns centrally
- **MCP servers**: if your organization already runs a code search or RAG index over the repository, expose it as an MCP tool so Claude queries it instead of reading files directly

See **server-managed or endpoint-managed settings** for how platform teams can enforce these centrally.

Recommend the right plugin at session start

Once conventions live in plugins, a teammate starting Claude in an unfamiliar part of the tree has no signal about which plugin that area's owners maintain. A `SessionStart` **hook** can close that gap, since anything the hook prints to stdout is added to Claude's context before the first prompt.

For example, you can write a script that reads the launch directory from the **hook input**, looks it up in a path-to-plugin map committed to the repository, and prints the recommendation for Claude to relay in its first reply. See **Automate actions with hooks** to write and register the hook.

Put it together

The combined configuration below uses the monorepo layout. The same files work for any subdirectory in a large single tree. Project settings load only from the directory you start Claude in, so each subdirectory's `.claude/settings.json` must be self-contained rather than layered on a root file.

The example commits `worktree`, `additionalDirectories`, and the `Read` deny rules in `.claude/settings.json` so every developer in `packages/api/` gets the same sibling access, sparse paths, and exclusions. The file below is the committed per-area settings for `packages/api/`:

packages/api/.claude/settings.json

```
{
  "worktree": {
    "sparsePaths": [
      ".claude",
      "packages/api",
      "packages/shared"
    ],
    "symlinkDirectories": [
      "node_modules"
    ]
  },
  "permissions": {
    "additionalDirectories": [
      "../shared"
    ],
    "deny": [
      "Read(./**/dist/**)",
      "Read(./**/build/**)"
    ]
  }
}
```

Because this session starts from `packages/api/`, sibling packages' CLAUDE.md files are already out of scope, so `claudeMdExcludes` is not needed here. Add it to the repository root's `.claude/settings.local.json` instead if you also start sessions from the root.

The `additionalDirectories` entry applies when you start Claude from `packages/api/` directly. Inside a worktree created from this session, the working directory is the worktree root, so this settings file does not load. The sibling packages are already reachable inside the worktree without it, but the deny rules need a second copy in the repository root's `.claude/settings.json` so worktree sessions pick them up, as the **worktree settings note** describes:

.claude/settings.json

```
{
  "permissions": {
    "deny": [
      "Read(./**/dist/**)",
      "Read(./**/build/**)"
    ]
  }
}
```

After setup, the repository has this layout:

```
monorepo/
  CLAUDE.md
  .claude/settings.json                          # deny rules
for worktree sessions
  packages/
    api/
      CLAUDE.md
      .claude/settings.json                      # worktree,
additionalDirectories, deny rules
      .claude/skills/api-testing/SKILL.md
    web/
      CLAUDE.md
      .claude/skills/component-patterns/SKILL.md
    shared/
      CLAUDE.md
```

With this setup, starting Claude from `packages/api/`:

- Loads the root CLAUDE.md and `packages/api/CLAUDE.md`, skips `packages/web/CLAUDE.md`
- Can read and edit files in `packages/api/` and `packages/shared/`
- Skips reads of build output under `dist/` and `build/` in `packages/api/`
- Has the api-testing skill available on demand
- Creates worktrees containing `.claude/`, `packages/api/`, `packages/shared/`, and root-level files, with the deny rules applied across the worktree from the root settings file

Scope and plan changes that span packages

The configuration above controls what Claude sees. When a single change touches several packages, such as updating a shared type along with every call site that uses it, how you scope and sequence the task also affects the result.

Two techniques help keep a cross-package change consistent:

- **Give Claude the whole change in one session**: handing over the shared edit and its call sites together keeps the decisions behind each edit consistent, rather than re-deriving them per package
- **Save the plan to a file before editing**: **plan first** and ask Claude to write the plan to a markdown file in the repository. A long cross-package session **compacts its context** along the way, and the saved plan survives where conversation history may not

Next steps

Once this configuration is in place, you can refine it:

- Use **hooks** to run per-directory linters or type-checkers after Claude edits files
- Review **Manage costs effectively** to understand how codebase size affects token usage and how to set spend limits before a wider rollout
- Read **How Claude Code works in large codebases** on the Claude blog for organizational rollout patterns and ownership models that sit above the per-repository configuration on this page